Here is a very detailed, deep-dive explanation of everything we have built in **Phase 1** so far.

As the Lead AI Agent Architect, your AI engine will rely heavily on these files, so understanding exactly what they do is critical. Phase 1 represents the **Data Acquisition and Gateway Layer** of the project.

---

1. browser_automation/viewport_manager.py

Purpose: Web applications look different depending on the device size. To find UI/UX bugs, we must test across multiple screen sizes automatically. **Functionality:**

- It uses Python's @dataclass(frozen=True) to create immutable configurations for **7 real-world devices** (e.g., desktop_1080p, mobile_standard which is an iPhone 12, tablet_landscape).
- It holds details like width, height, and device_scale_factor (for retina displays).
- It has a helper function to_playwright_context_options() which converts these configurations into the exact dictionary format that the Playwright engine needs to simulate the device.

2. browser_automation/browser_engine.py

Purpose: This is the heavy lifter. It acts as the "eyes" of your application, navigating to URLs and extracting both visual and structural data. **Functionality:**

- **capture_page()**: It launches an asynchronous, headless Chromium browser. For a given URL, it loops through the 3 default viewports (Desktop, Tablet, Mobile). For each one, it simulates the device, waits for the network traffic to settle (so React/Vue apps can finish rendering), takes a **full-page screenshot**, and grabs the **raw HTML**.
- **extract_computed_styles()**: This is incredibly important for Friend 2's WCAG contrast checker. It doesn't just read the HTML; it injects a JavaScript snippet directly into the browser to calculate the *final, rendered CSS* of every text element on the page. It returns the exact color, background-color, font-size, and bounding box (X, Y coordinates) of the text.

3. browser_automation/sandbox_injector.py

Purpose: When your AI agent (Phase 3) generates a CSS patch to fix a UI bug, we cannot just assume the AI is correct. We need a "sandbox" to test the AI's fix safely. **Functionality:**

- **apply_and_verify()**: It takes a URL and a CSS patch string (e.g., body { background: white; }).
- It loads the original page and takes a "before" screenshot.
- Then, it uses Playwright to dynamically inject a <style> tag containing the AI's CSS patch directly into the page's <head>.
- It takes an "after" screenshot and returns both images side-by-side. Your AI Agent will eventually use these two images to visually verify if the fix worked!

4. api_backend/main_api.py

Purpose: This is the FastAPI gateway. It acts as the traffic controller. When Friend 3 builds the Streamlit UI, the UI will send HTTP requests to this API. **Functionality:**

- **GET /health**: A simple endpoint to check if the API and the Redis database are online.
- **POST /audit-url**: The main endpoint. It accepts a JSON payload with a url. It orchestrates the browser_engine.py to capture the screenshots and extract the computed styles, packages them into a clean JSON response, and sends it back to the client.
- **POST /sandbox/verify-patch**: An endpoint that exposes the sandbox_injector.py to the outside world, allowing clients to test CSS patches over HTTP.

5. api_backend/redis_cache.py

Purpose: Playwright is powerful, but launching a browser and rendering a webpage is slow (takes 5-10 seconds). If two users audit the exact same URL, we shouldn't waste time running Playwright twice. **Functionality:**

- It connects to a Redis database to cache (save) the results of an audit.
- **SHA-256 Hashing:** To save memory, it hashes the requested URL (e.g., https://google.com becomes 2d711642b726...) and uses that as the database key.
- **Lazy Connection & Graceful Degradation:** It's designed very robustly. It doesn't try to connect to Redis until it actually needs to. If the Redis server crashes or is turned off, the application *will not crash*. It will simply print a warning, bypass the cache, and run Playwright directly.

6. Infrastructure (Dockerfile & docker-compose.yml)

Purpose: Running headless browsers requires specific system libraries (like libnss3 or libglib2.0). If your friends try to run this on their different laptops

(Windows, Mac, Linux), it might crash because they are missing system files. **Functionality:**

- **Dockerfile**: Packages the entire API and Playwright browser into an isolated, standardized Linux environment.
- **docker-compose.yml**: Defines two services: our visionguard-api and the redis database. It networks them together and ensures the API waits for the Redis database to fully boot up before starting.

Summary of How It All Connects Right Now:

If someone sends a request to your API right now: POST /audit-url {"url": "https://example.com"}, the API (main_api.py) checks Redis (redis_cache.py). If it's not cached, the API tells Playwright (browser_engine.py) to open Chromium. Playwright navigates to the URL, takes 3 screenshots at different device sizes (viewport_manager.py), and injects JS to steal the final CSS colors. The API saves this data in Redis, and sends the screenshots and CSS data back to the user.